

Implementation Assignment 2

Pseudo Code

For each pair of sequences (s_1 , s_2):

- Build a 2D table where each cell represents the minimum cost to align the prefixes of s_1 and s_2 up to that point.

- Fill in the edges first, the cost of aligning each prefix against nothing but gaps.

- Then fill in the rest of the table left-to-right, top-to-bottom.

- At each cell, consider three options and pick the cheapest:

 - align the two characters against each other (diagonal)

 - align s_1 's character against a gap (up)

 - align s_2 's character against a gap (left)

- The bottom-right corner now holds the optimal cost.

- Trace back through the table from bottom-right to top-left, following whichever choice was made at each step, to reconstruct the actual alignment.

- Output the two aligned strings and the total cost.

Asymptotic Analysis

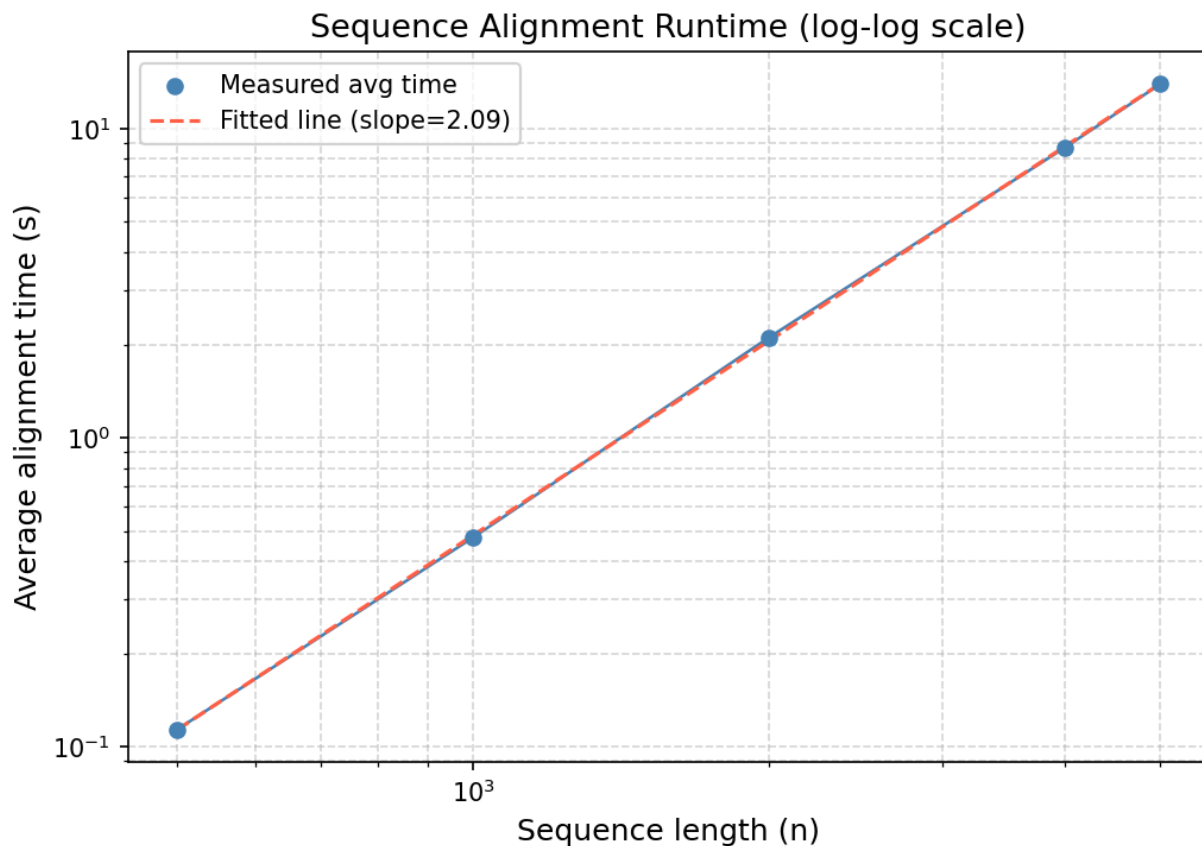
We create a table that is $m+1$ by $n+1$. The first step is filling the first row and column, which is $O(m+n)$. We then fill the remaining squares, which requires $O(m*n)$. Then we traceback from the bottom right, and at worst we travel to the top right, which is $O(m+n)$, or the entire length plus width of the table.

The dominant step is filling the cells at $O(m*n)$. In our case, m and n are the same size, so that means the run time complexity of our align function is $O(n^2)$.

Empirical Runtime

I used my laptop to test the runtime for different lengths. I wrote a python script that generates 10 random pairs for each length, and then I time how long it takes for my align function to run. Then we average out the resulting times and print them to the terminal. We are not including the outputting of the results to a file in the run time, we are only aligning and calculating the cost for each pair.

Length	Avg Time (s)	Min (s)	Max (s)
500	0.1139	0.1057	0.1457
1000	0.4762	0.4611	0.4945
2000	2.1145	2.0341	2.2158
4000	8.7031	8.2766	9.1099
5000	13.9582	13.6962	14.3367



Assumed complexity is again $O(n^2)$

Interpretation and Discussion

Our asymptotic analysis said that the runtime would scale quadratically. When we ran the benchmark and plotted the results, we put it into a logarithmic scaled graph. Plotting polynomial functions on a logarithmic scaled graph always results in the points being placed on a linear line. The slope of this line is equivalent to the power of the polynomial. Since we only used 5 averaged out data points, there is still a margin of error, and python probably had some overhead, but our resulting slope was 2.09. This is incredibly close to 2, which corresponds to a quadratic function. This means we were absolutely right to say the runtime complexity is $O(n^2)$